

INTRODUCTION TO SYSTEMS ANALYSIS AND DESIGN: AN AGILE, ITERATIVE APPROACH

SATZINGER | JACKSON | BURD

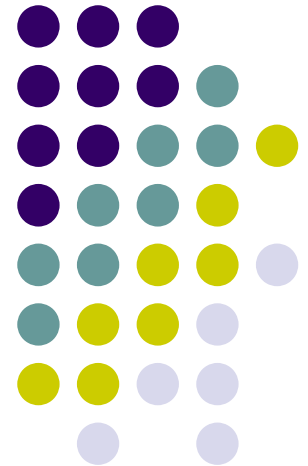
CHAPTER 8

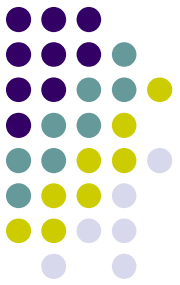
Approaches to System Development

Chapter 8

Introduction to Systems
Analysis and Design:
An Agile, Interactive Approach
6th Ed

Satzinger, Jackson & Burd

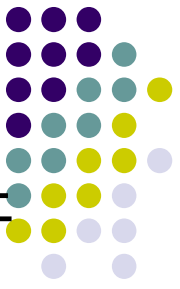




Chapter 8 Outline

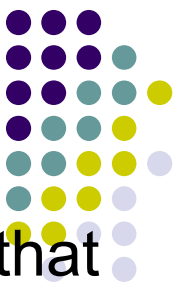
- The Systems Development Life Cycle (回顾系统开发生命周期)
- The Support Phase of the SDLC (支持阶段)
- 详细设计阶段的目的与任务
- Two Approaches to Software Construction and Modeling (软件构造与建模)
- Agile Development (敏捷开发)

Learning Objectives

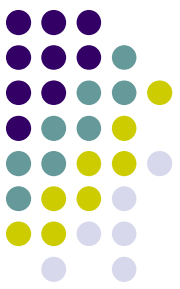


- Describe the system development life cycle (系统开发生命周期)
- Describe the key activities and tasks of information system support (信息系统支持的关键活动和任务)
- Explain what comprises a system development methodology—the SDLC as well as models, tools, and techniques (系统开发方法中包括哪些内容)
- Describe the two overall approaches used for software construction and modeling: the structured approach and the object-oriented approach (描述用于软件设计和建模的两种方法：结构化和面向对象)
- Describe the key features of Agile development (敏捷开发的特征)

Overview



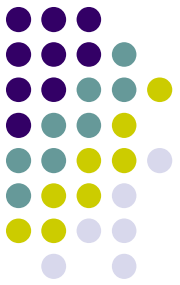
- Chapter 1 demonstrated a system development project that used an iterative and agile system (迭代和敏捷系统) development life cycle (SDLC)
- Later chapters focused on Systems Analysis activities (分析活动) and tasks and some System Design activities (系统设计活动) and tasks
- Now we return to look at the SDLC and related concepts in more detail
 - Predictive versus Adaptive SDLC variations (预测性 vs 适应性)
 - Activities and Tasks of System Support (活动和任务)
 - Impacts of Traditional versus OO development (传统和面向对象)



Chapter 8 Outline

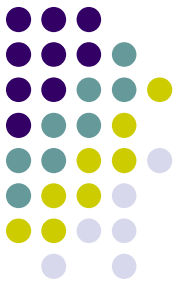
- The Systems Development Life Cycle (回顾系统开发生命周期)
- The Support Phase of the SDLC
- 详细设计阶段的目的与任务
- Two Approaches to Software Construction and Modeling
- Agile Development

The System Development Life Cycle (SDLC)

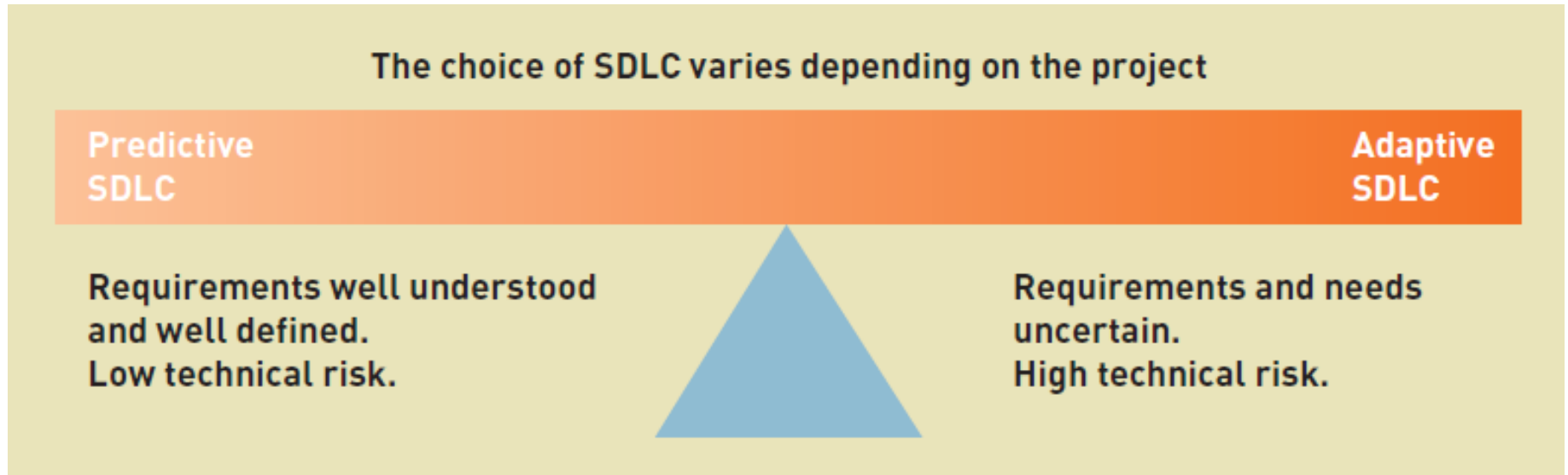


- There are two general approaches to the SDLC
- Predictive Approach (预测方法)
 - Waterfall model (瀑布模型)
 - Assumes the project can be planned in advance and that the information system can be developed according to the plan
 - Requirements are well understood and/or low technical risk
- Adaptive Approach to the SDLC (适应方法)
 - Iterative model (as see in this text) (迭代模型)
 - Assumes the project must be more flexible and adapt to changing needs as the project progresses
 - Requirements and needs are uncertain and/or high technical risk

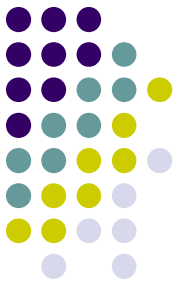
The System Development Life Cycle (SDLC)



- Most projects fall on a continuum between Predictive and Adaptive

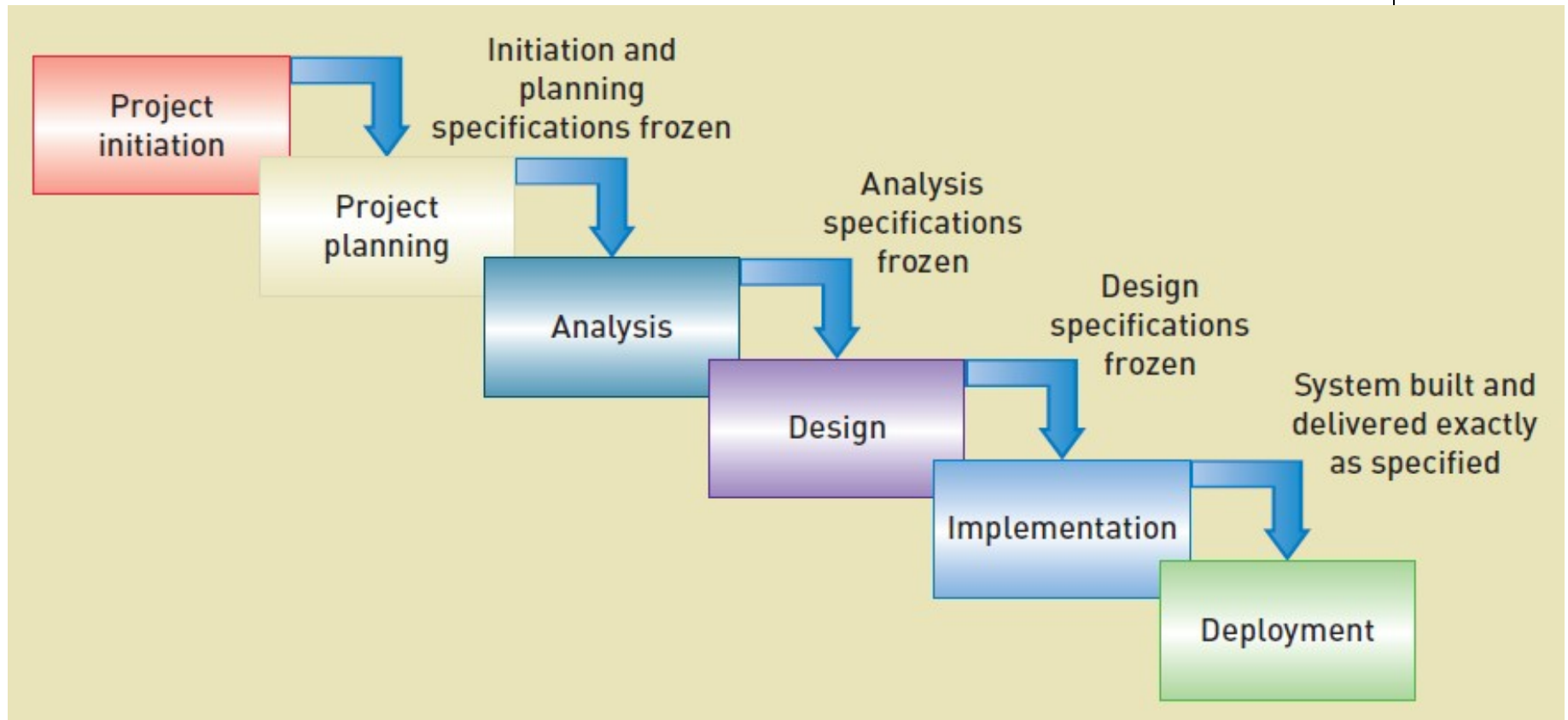
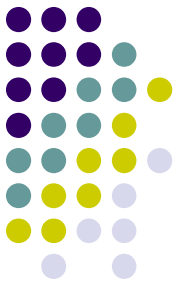


Traditional Predictive SDLC



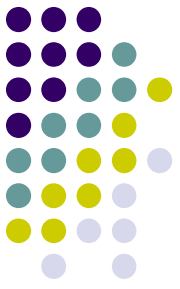
- Earlier approach (早期方法) based on engineering
- Typically have sequential Phases (通常由连续的阶段)
 - planning, analysis, design, implementation, and deployment (计划、分析、设计、开发、部署)
- Waterfall model (瀑布模型)
 - SDLC that assumes phases can be completed sequentially with no overlap or iteration (没有重叠或迭代)

Traditional Predictive SDLC

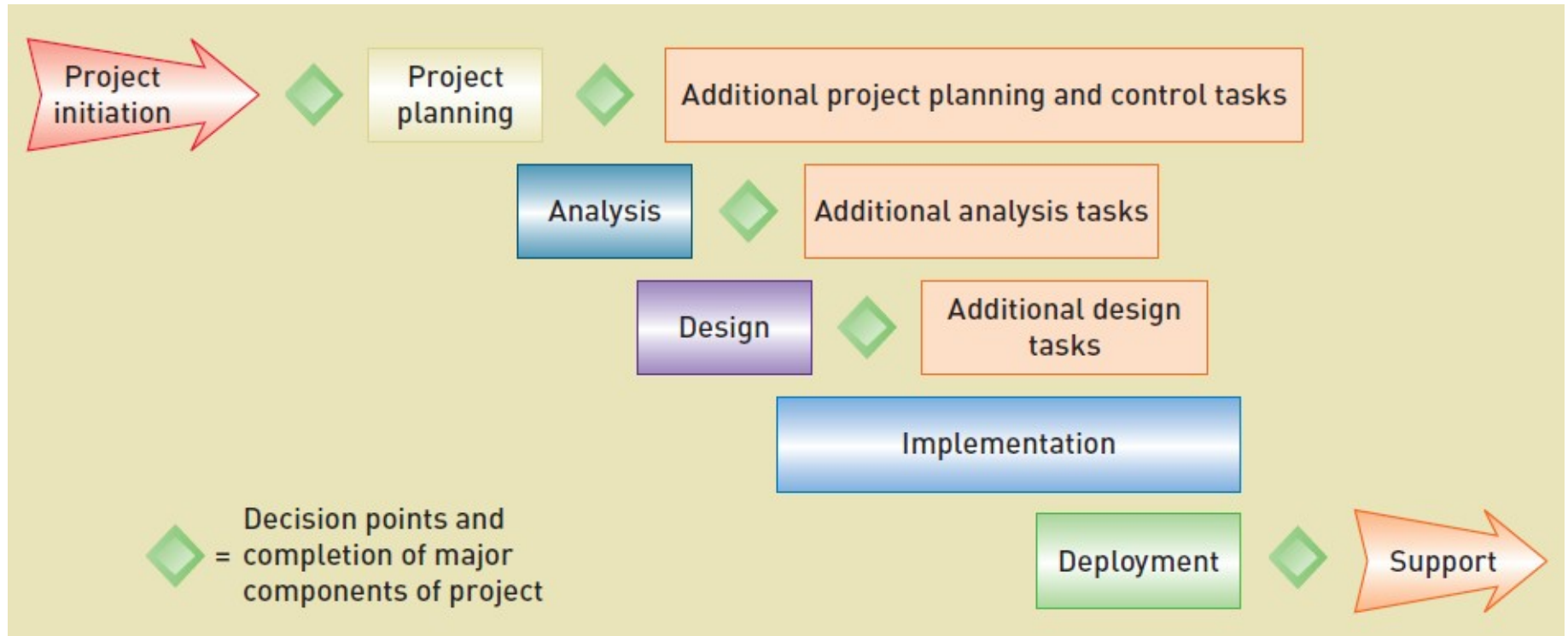


Newer Overlapping Phases

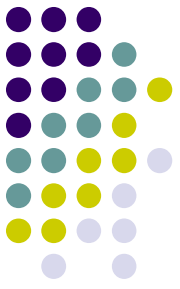
Predictive SDLC



- More flexibility, but still assumes predictive planning and sequential phases



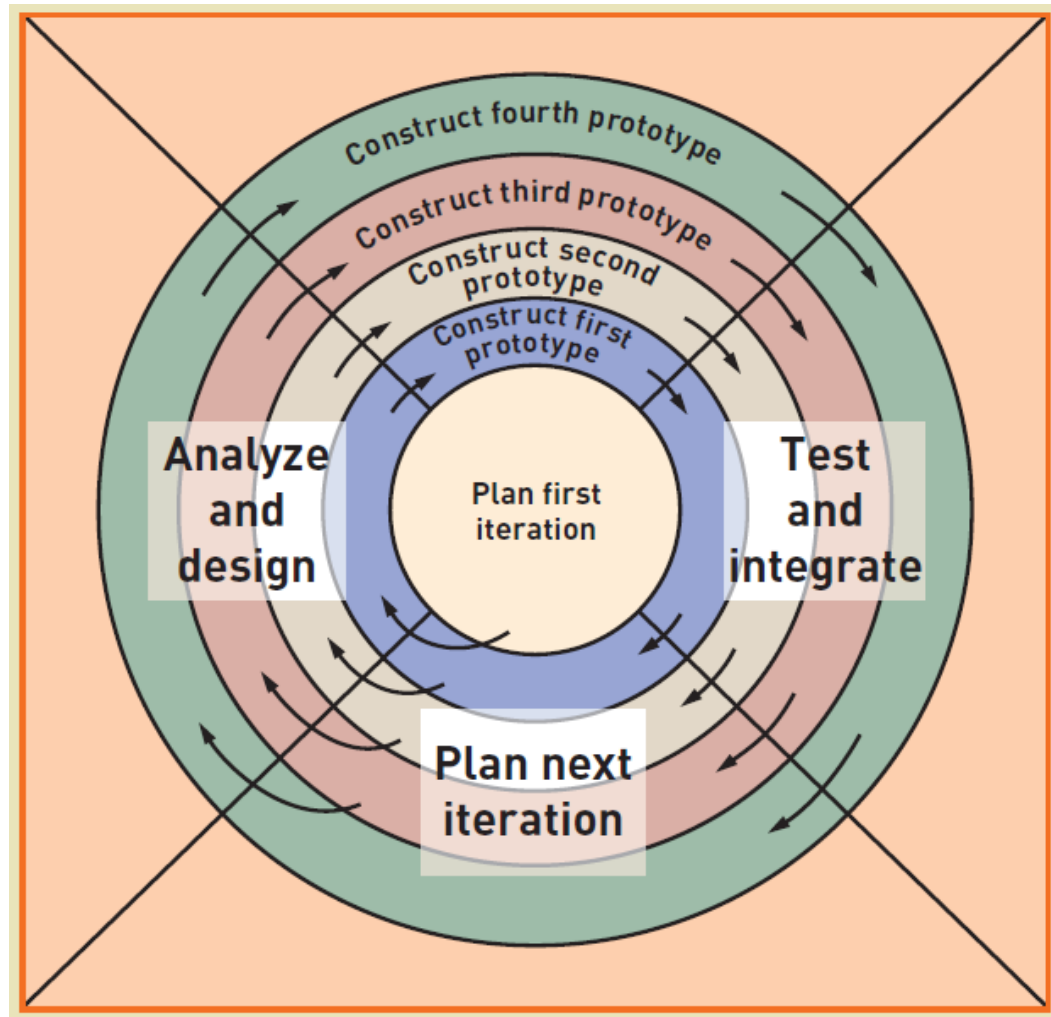
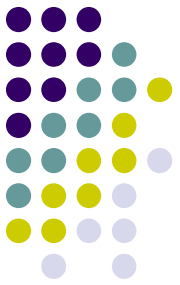
Newer Adaptive SDLC



- Emerged in response to increasingly complex requirements (日益复杂的需求) and uncertain technological environments (不确定的技术环境)
- Always includes **iterations** (迭代) where some of design and implementation is done from the beginning

Spiral Model

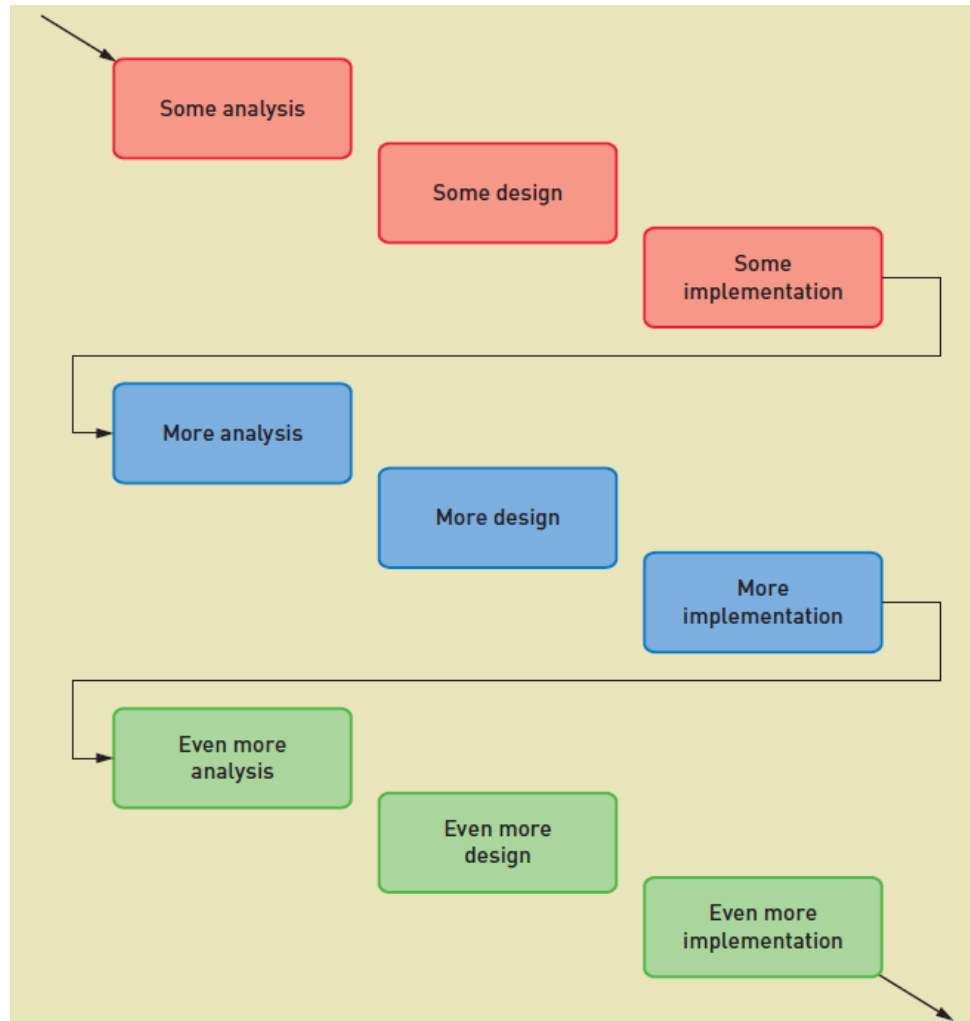
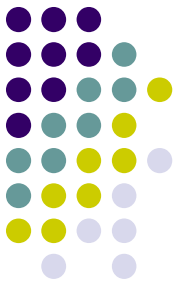
The First Adaptive SDLC



Introduction to Systems Analysis and Design, 6th Edition

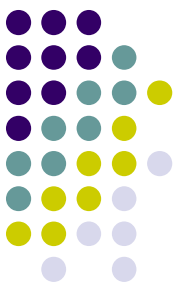
Iterative Model

Popular Way to Represent Adaptive SDLC



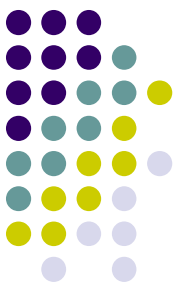
Introduction to Systems Analysis and Design, 6th Edition

14



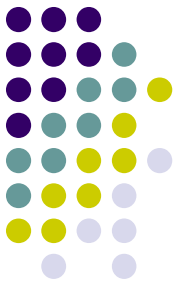
Chapter 8 Outline

- The Systems Development Life Cycle (SDLC)
- The Support Phase of the SDLC (支持阶段)
- 详细设计阶段的目的与任务
- Two Approaches to Software Construction and Modeling
- Agile Development



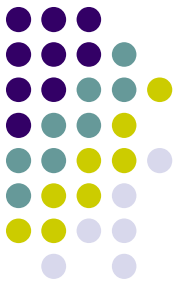
The SDLC Support Phase

- All information systems need to be supported once completed (所有信息系统一旦完成都需要得到支持)
- Predictive SDLCs typically include support as a project phase (预测性的系统开发生命周期明确地包括一个支持阶段)
- Support Activities (支持活动)
 - Activities whose objective is to maintain and enhance the system after it is installed and in use (维护系统、支持用户)



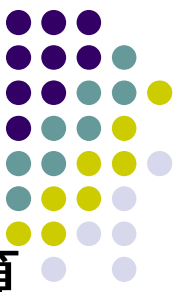
Support Activities

- Maintaining the system (维护)
 - Fix problems/error (修复错误)
 - Make minor adjustments (小的调整)
 - Update for changes in operating systems or environments (针对系统更改进行更新)
- Enhancing the system (增强)
 - Add desired functionality (添加所需功能)
 - Add or change functionality to comply with regulations or legislation (添加更改功能)
- Supporting the users (支持用户)
 - Ongoing user training (用户培训)
 - Help desk (帮助台)



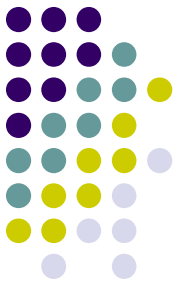
Chapter 8 Outline

- The Systems Development Life Cycle (SDLC)
- The Support Phase of the SDLC
- 详细设计阶段的目的与任务
- Two Approaches to Software Construction and Modeling
- Agile Development

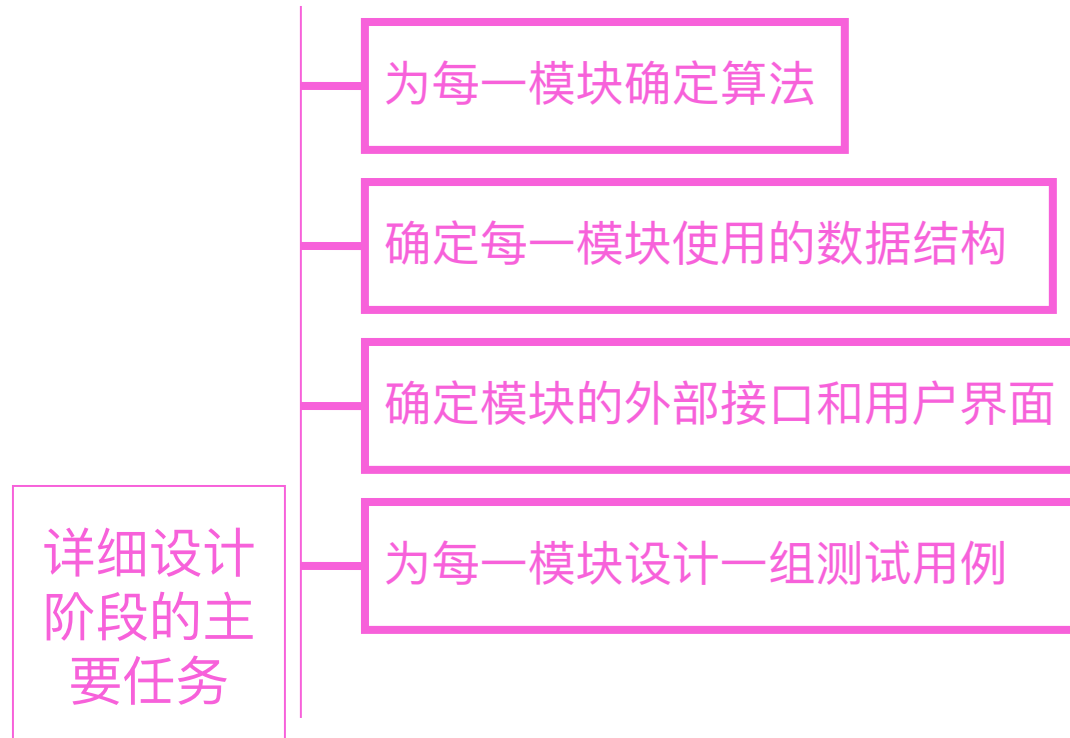


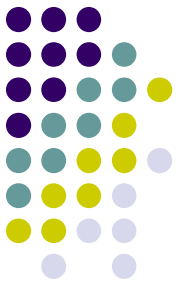
详细设计阶段的目的与任务

- 详细设计的目的：为每一个模块确定采用的算法和模块内数据结构，用某种选定的表达工具给出清晰的描述。
- 详细设计阶段的主要任务：编写软件的“详细设计说明书”



详细设计阶段的目的与任务

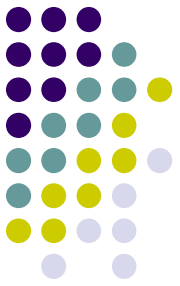




Chapter 8 Outline

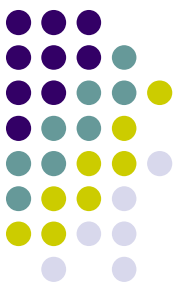
- The Systems Development Life Cycle (SDLC)
- The Support Phase of the SDLC
- 详细设计阶段的目的与任务
- Two Approaches to Software Construction and Modeling (软件构建和建模的两种方法)
- Agile Development

Two Approaches to Software Construction and Modeling

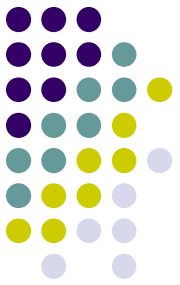


- The Structured Approach (结构化方法)
 - Earlier approach. Assumes a system is a collection of processes that interact with data
 - Structured analysis, structured design, and structured programming
- The Object-Oriented Approach (面向对象方法)
 - More recent approach. Assumes a system is a collection of objects that interact to complete tasks
 - OO analysis, OO design, and OO programming

Two Approaches to Software Construction and Modeling

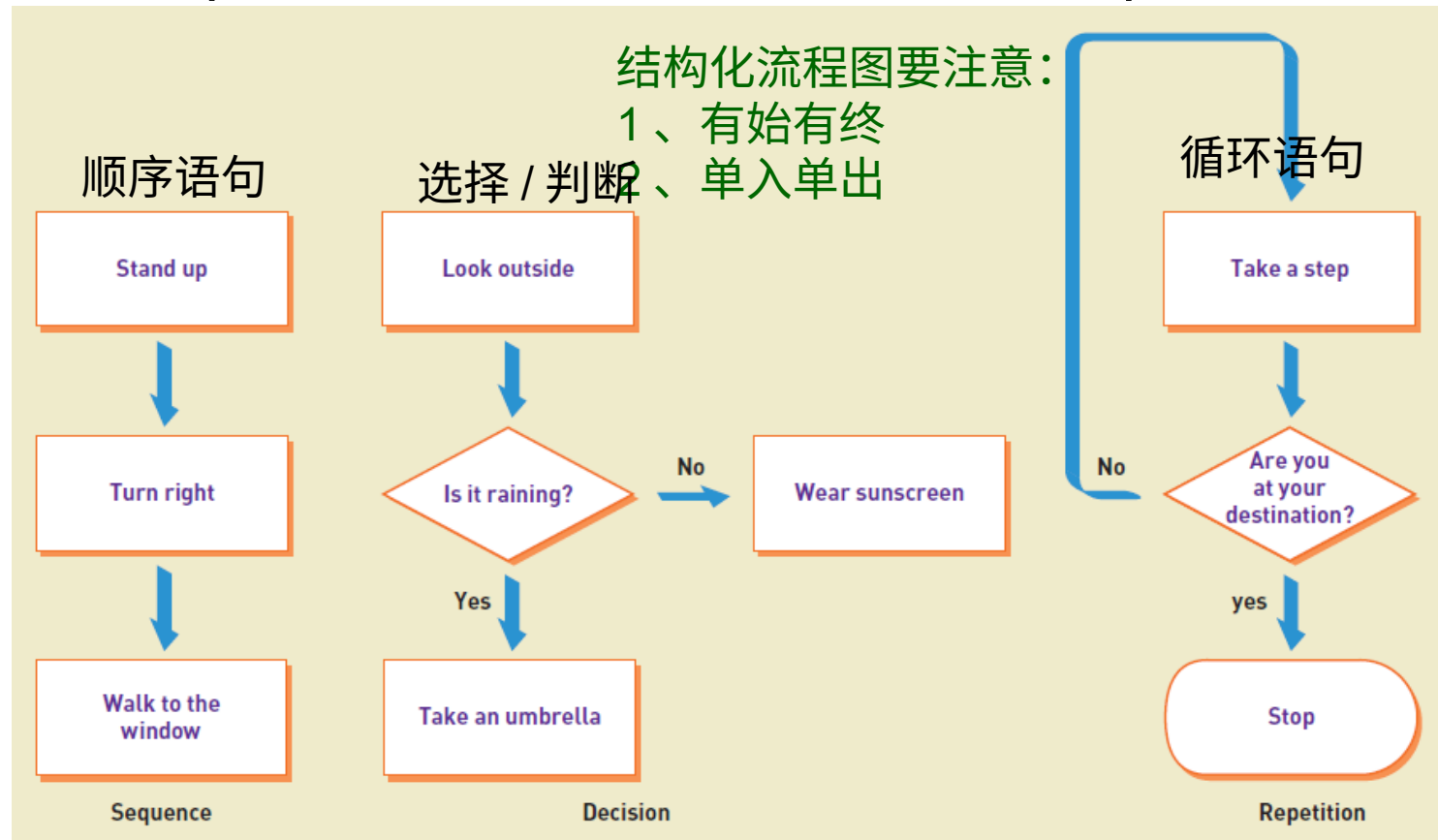


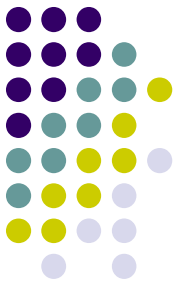
- The Structured Approach (结构化方法)
 - 1965 年结构程序设计的概念最早由 E.W.Dijkstra 提出：程序的质量与程序中所包含的 GO TO 语句的数量成反比
 - 1966 年 Bohm 和 Jacopini 证明了只用“顺序”、“选择”和“循环”控制结构就能实现任何单入口单出口的程序。
 - 结构化设计方法是基于模块化、自顶向下细化、结构化程序设计等程序设计技术基础发展起来的。
 - 它所提供的方法和原则，主要是用来指导软件的概要设计。它还提供了一种“结构图”的描述工具，是专门用来描述软件的总体结构的。



1. 结构化方法

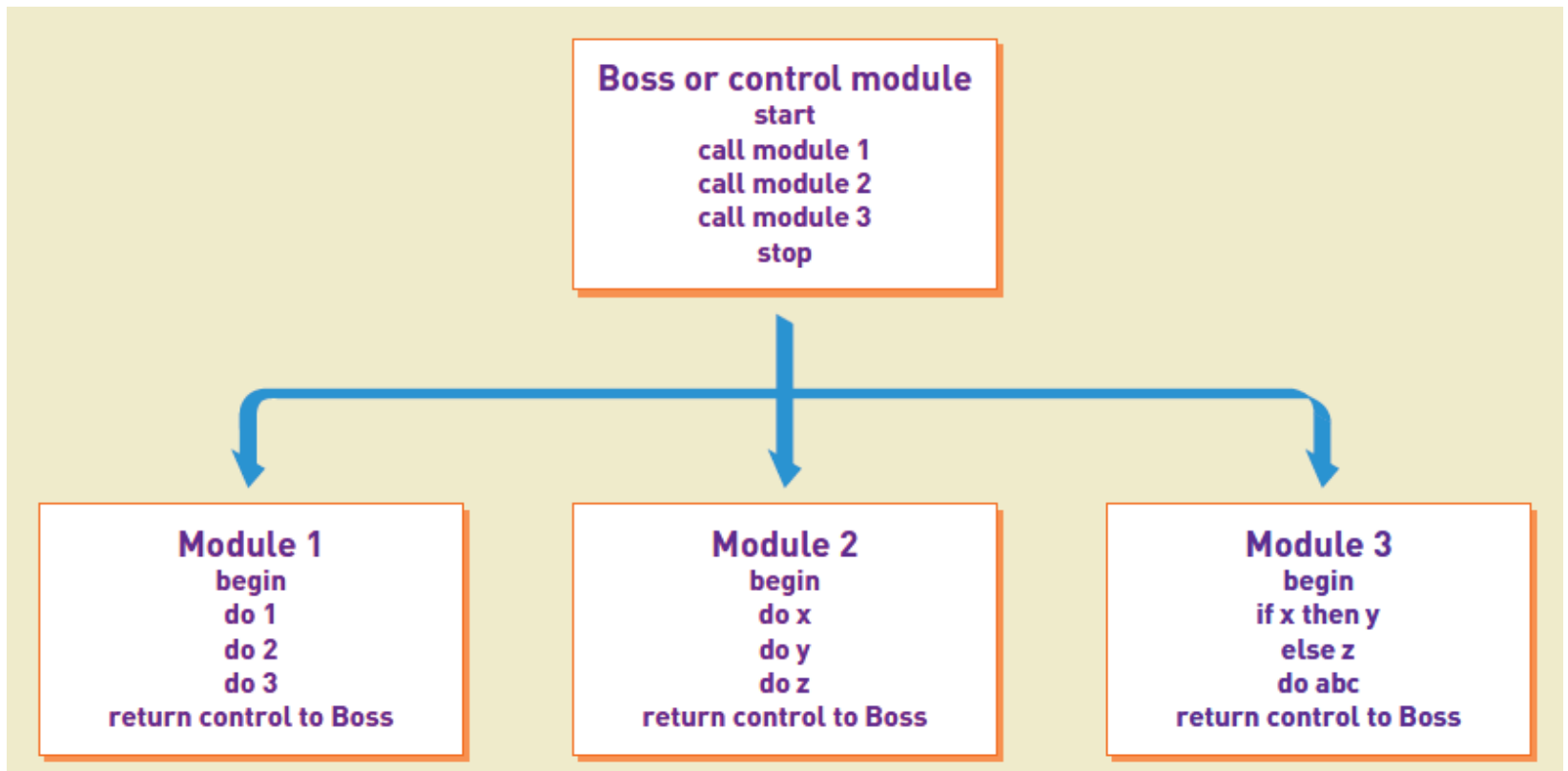
- Structured Programming
 - Sequence, selection/decision, and repetition

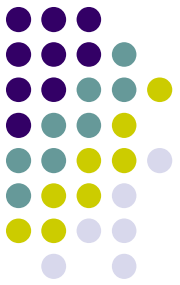




1. 结构化方法

- Top down, modular programming
- 自上而下，模块化设计





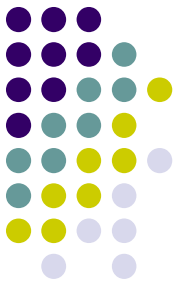
1. 结构化方法

结构化设计的目的

使程序的结构尽可能反映要解决的问题的结构

结构化设计的任务

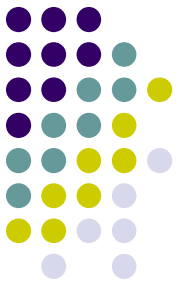
完成目标系统的一系统结构图 (SC)



1. 结构化方法

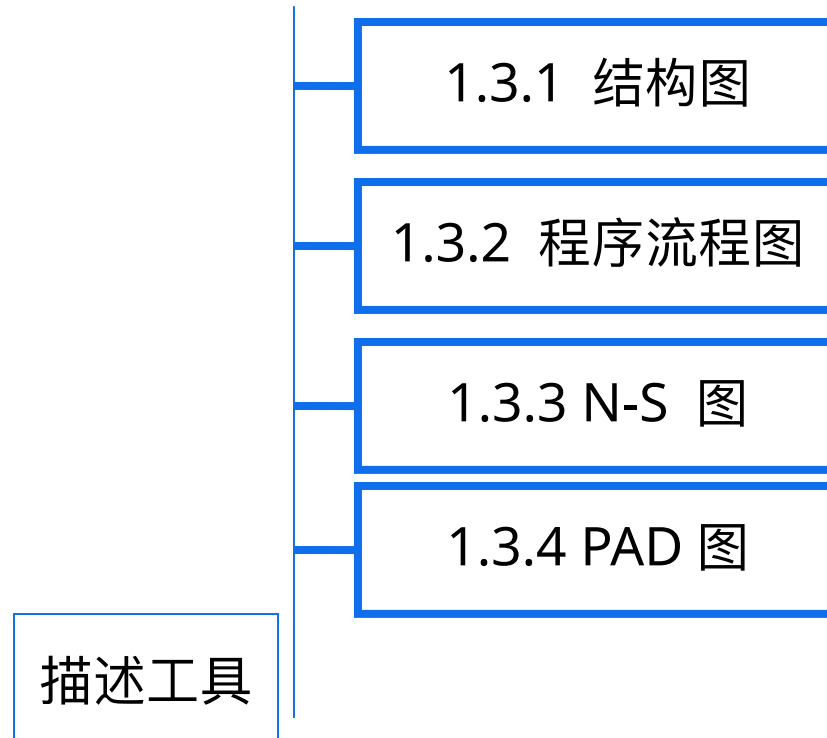
1.2 结构化程序设计原则

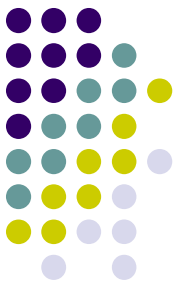
- 1、使用语言中的顺序、选择、重复等有限的基本控制结构表示程序
- 2、选用的控制结构只准许有一个入口和一个出口
- 3、程序语句组成容易识别的块（Block），每块只有一个入口和一个出口
- 4、复杂结构应该用基本控制结构进行组合嵌套来实现
- 5、严格控制 GOTO 语句



1. 结构化方法

1.3 图形描述工具

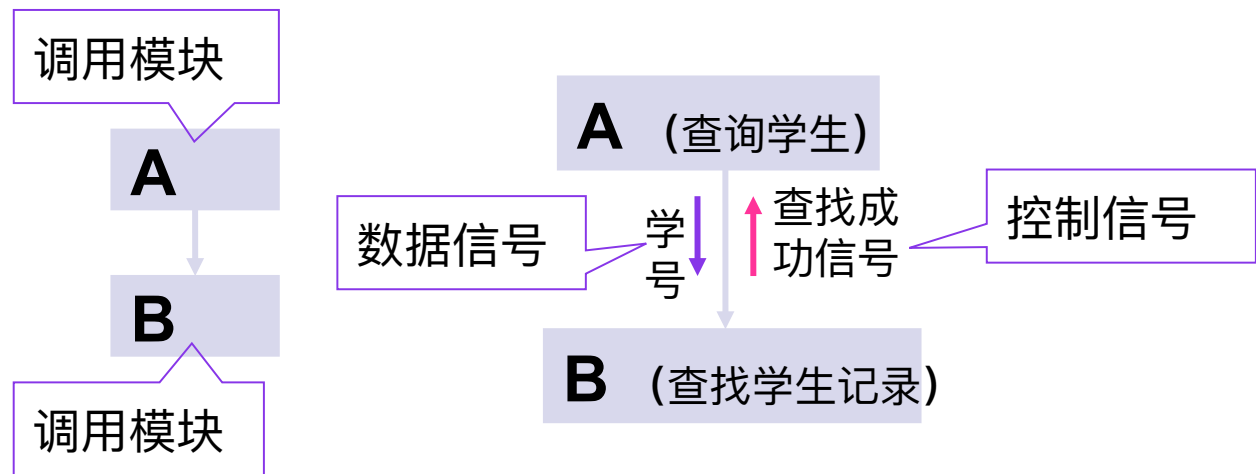




1. 结构化方法

1.3.1 结构图（ Structured Chart ）：

- 该图常用来表示系统的软件结构。可以清楚地表达软件结构中模块间的层次调用关系和模块之间的联系。
- 主要符号：
 - 模块 -- 在 SC 图中用矩形框表示，并用名字来标记它
 - 模块的调用关系和接口



-- 模块调用关系

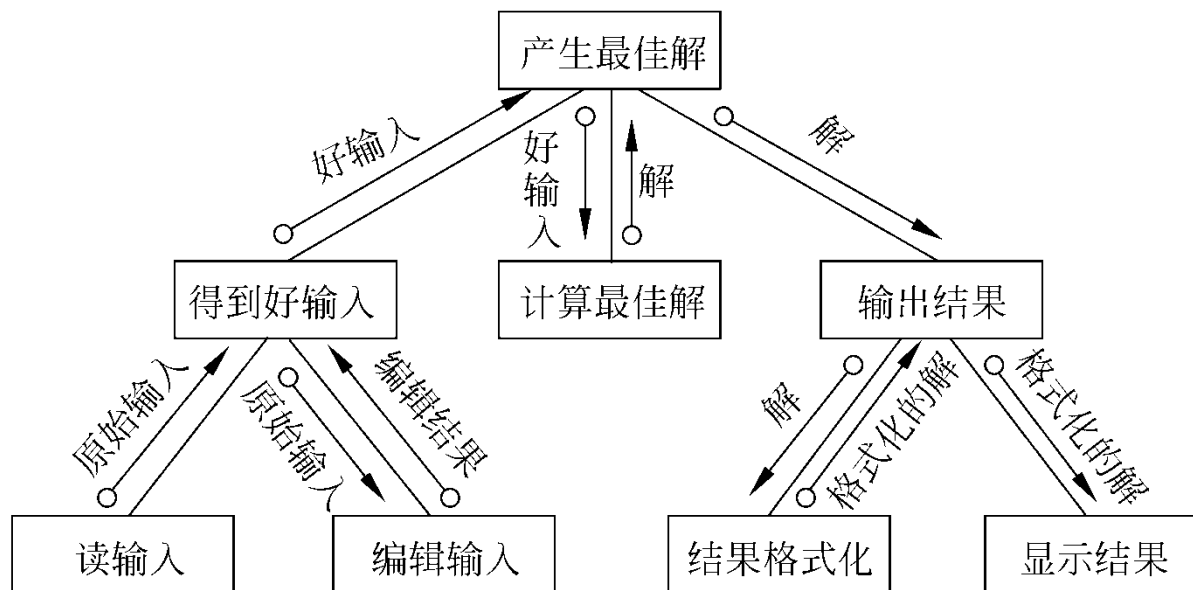
-- 模块间接口的表示

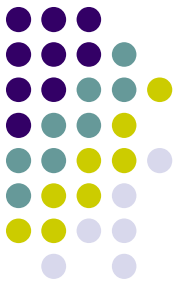


1. 结构化方法

1.3.1 结构图（ Structured Chart ）：

Yourdon 提出的结构图是进行软件结构设计的工具。图中一个方框代表一个模块，框内注明模块的名字或主要功能；方框之间的箭头（或直线）表示模块的调用关系。尾部是空心圆表示传递的是数据，实心圆表示传递的是控制



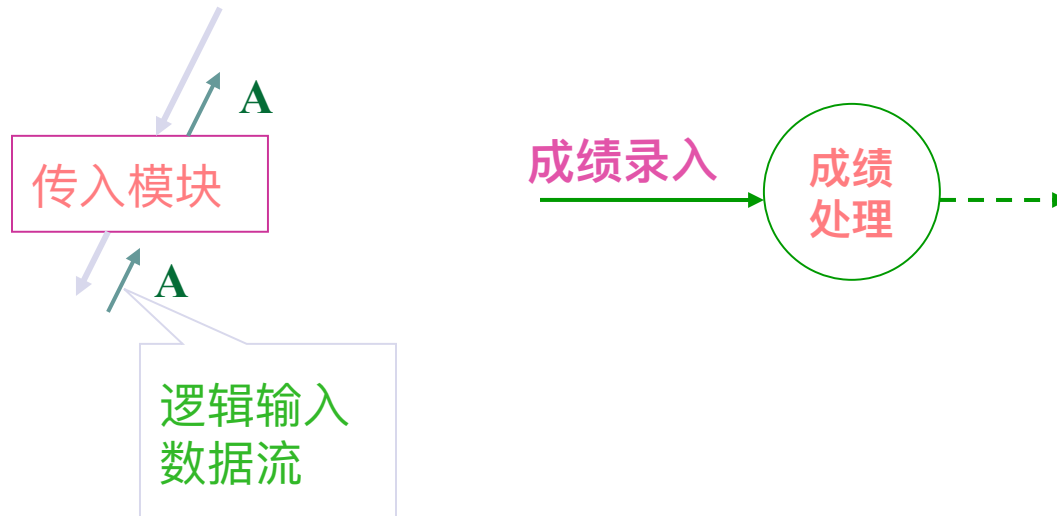


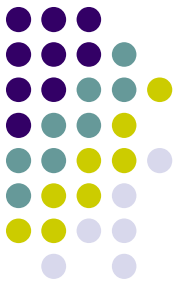
1. 结构化方法

1.3.1 结构图

原子模块：在系统结构图中通常是指不能再分割的底层模块

传入模块：从下属模块取得数据，进行某些处理，再将其结果传给上级模块。在此，将它传送的数据流称为逻辑输入数据流。

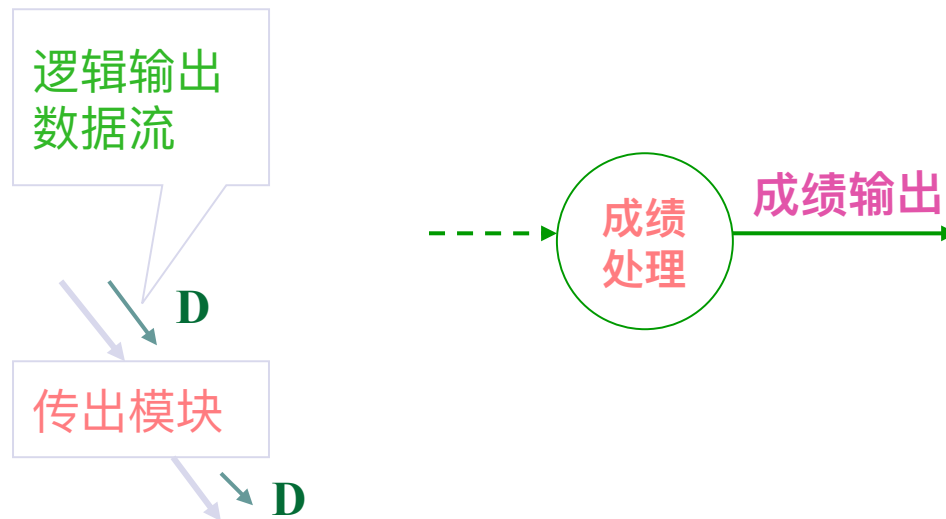


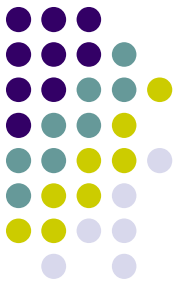


1. 结构化方法

1.3.1 结构图

传出模块：从上级模块获得数据，进行某些处理，再将其结果传给下属模块。在此，将它传送的数据流称为逻辑输出数据流。



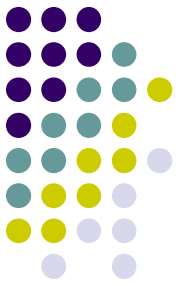


1. 结构化方法

1.3.1 结构图

•变换模块：也叫加工模块。它是从上级模块获得数据，进行特定的处理，将其转换为其他形式，再传回上级模块它所加工的数据流叫做变换数据流。

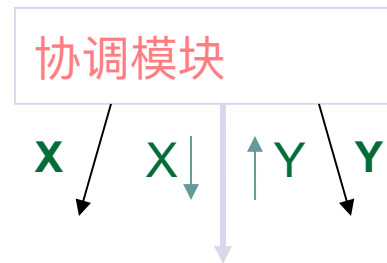


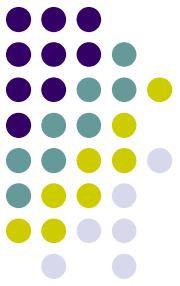


1. 结构化方法

1.3.1 结构图

•协调模块：对所有下属模块进行协调和管理的模块。
在一个好的系统结构图中，协调模块应在较高层出现。

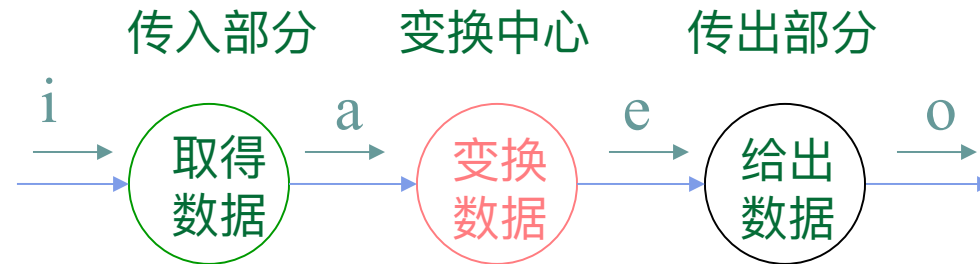




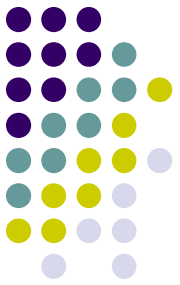
1. 结构化方法

1.3.1 结构图

- 变换型系统结构图：



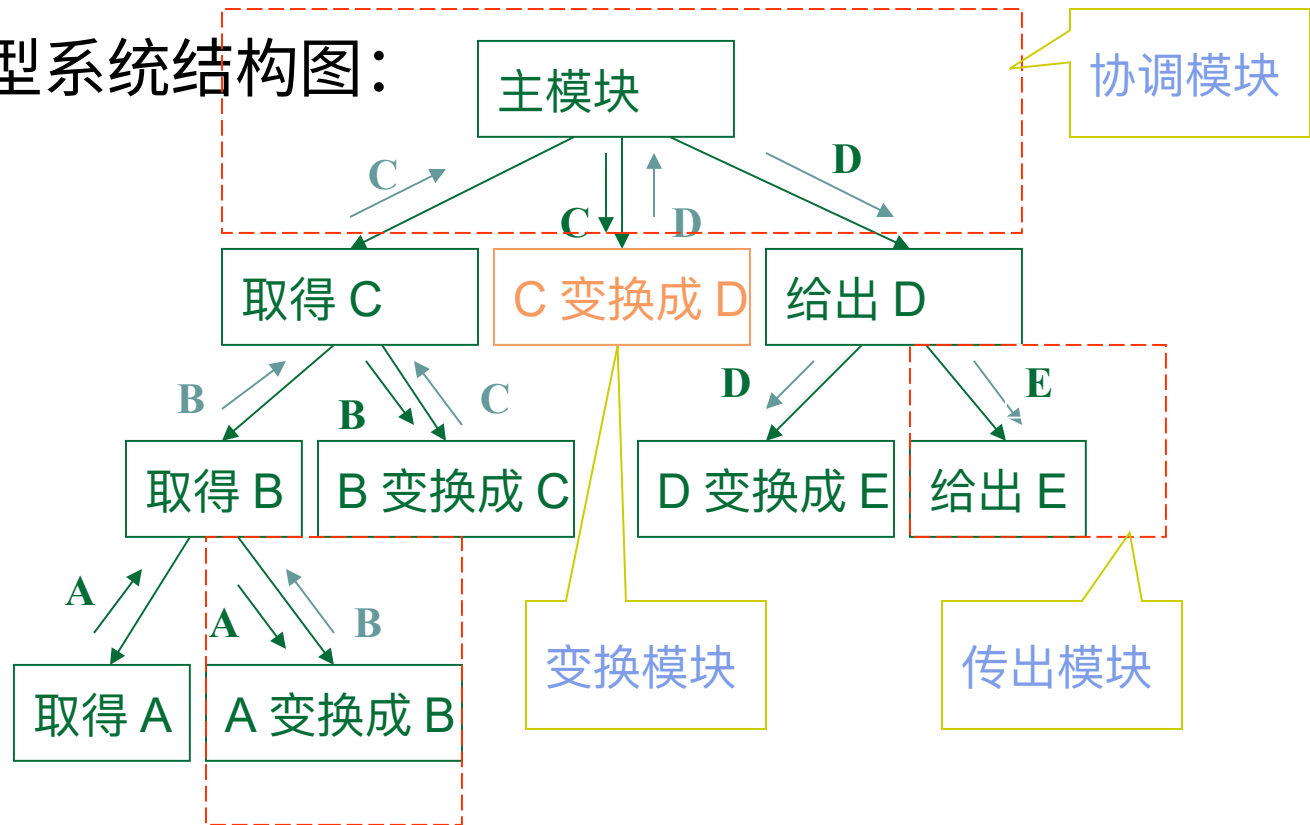
----- 具有变换型数据流图



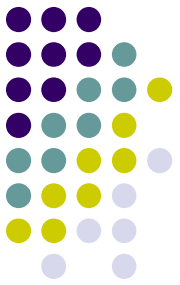
1. 结构化方法

1.3.1 结构图

• 变换型系统结构图：



----- 具有变换型系统结构图

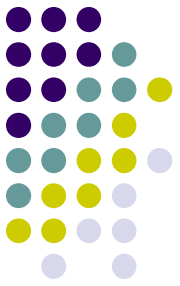


1. 结构化方法

1.3.1 结构图

- 事务型系统结构图：

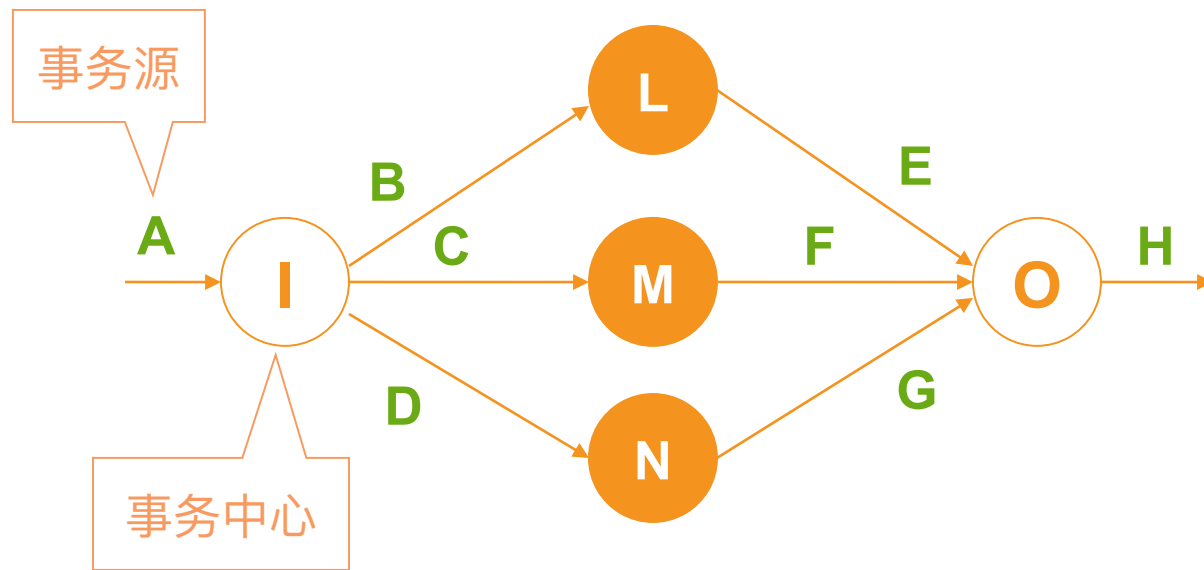
- 事务：引起、触发或启动某一动作或一串动作的任何数据、控制信号、事件或状态的变化
- 由它接受一项事务，根据事务处理的特点和性质选择分配一个适当的处理单元，然后给出结果



1. 结构化方法

1.3.1 结构图

- 事务型数据流图：

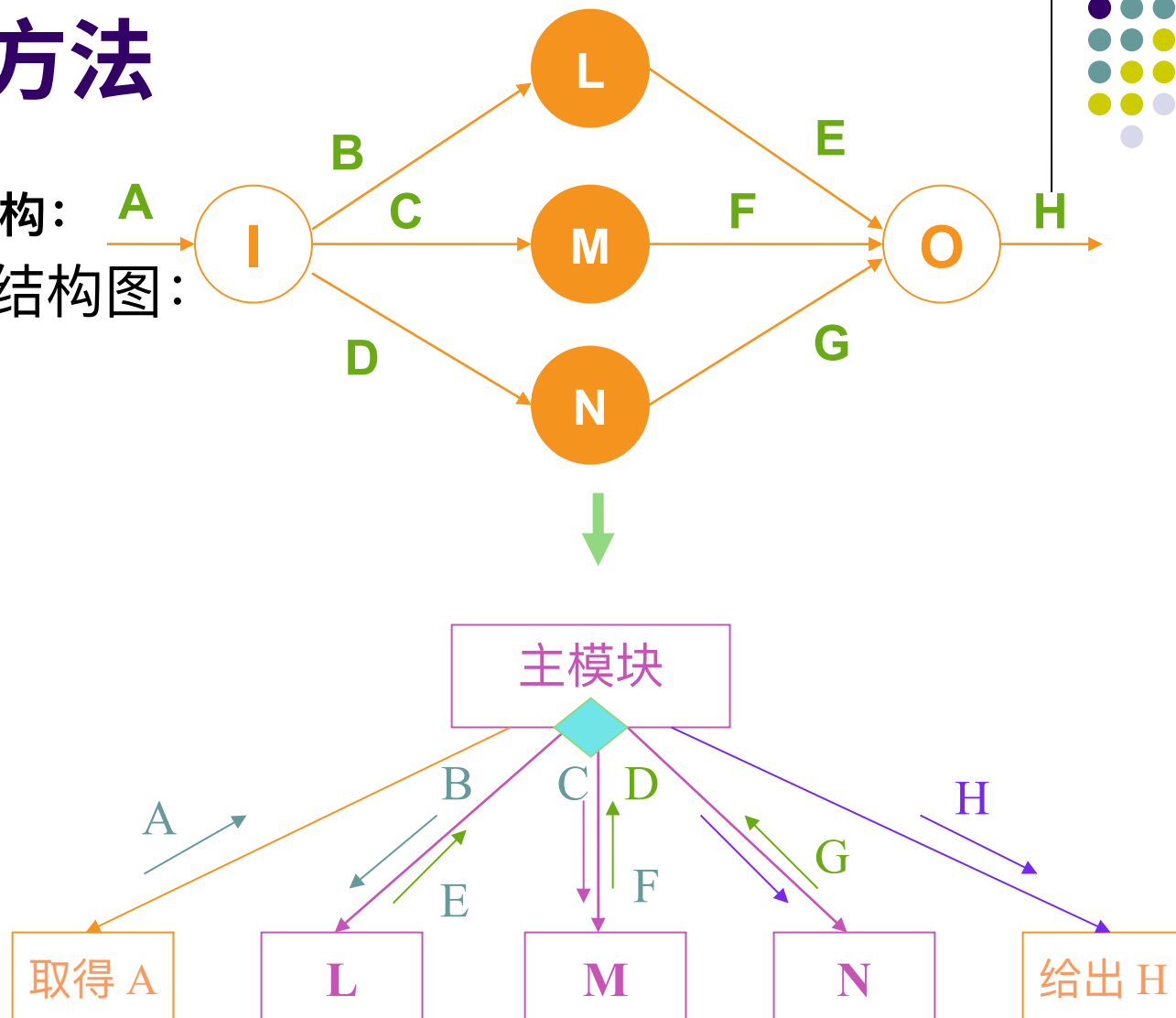


----- 事务型 数据流图

1. 结构化方法

1.2 典型的系统结构：

- 事务型结构图：



1. 结构化方法 - 例题



请将下列给出的采购子系统 **数据流图** 转换成 **系统结构图**

加工名称：

2.1 按书号汇总缺书

2.2 按出版社汇总缺书

2.3 修改教材库存和待购量

文件名称：

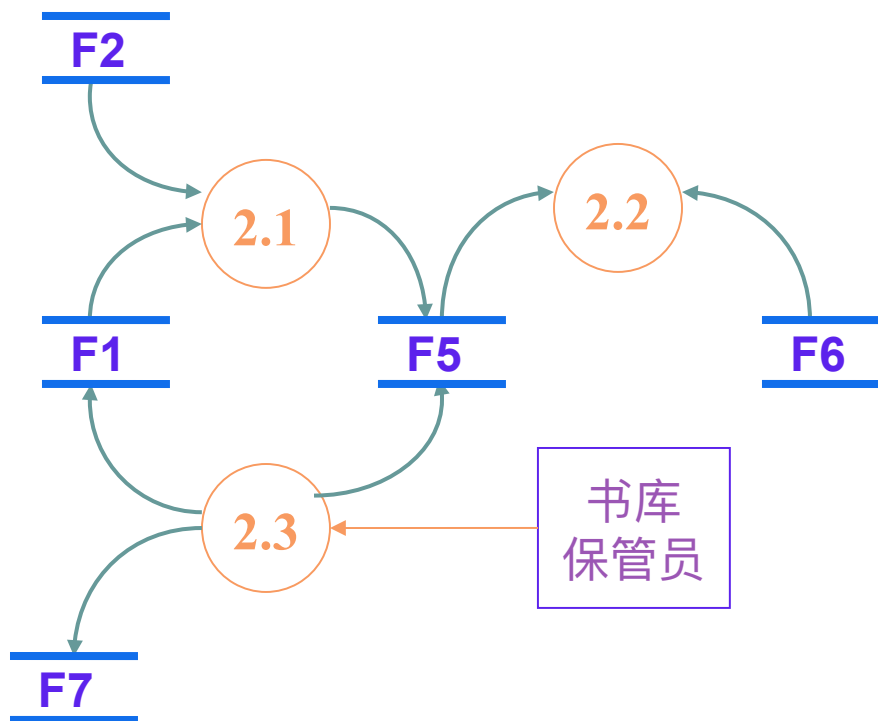
F1 教材存量表

F2 缺书登记表

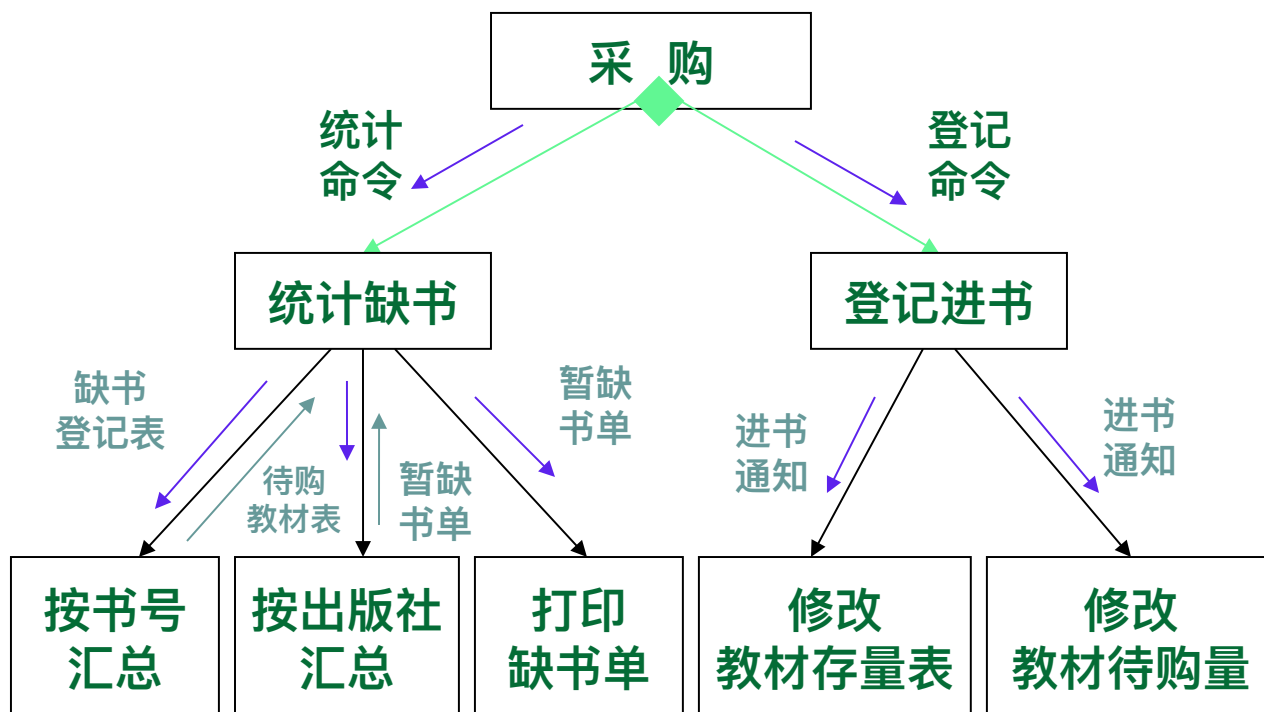
F5 待购教材表

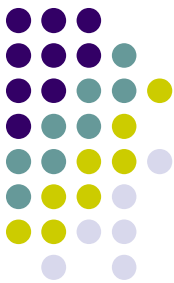
F6 教材一览表

F7 进书登记表



1. 结构化方法 - 例题





1. 结构化方法

1.3.2 程序流程图

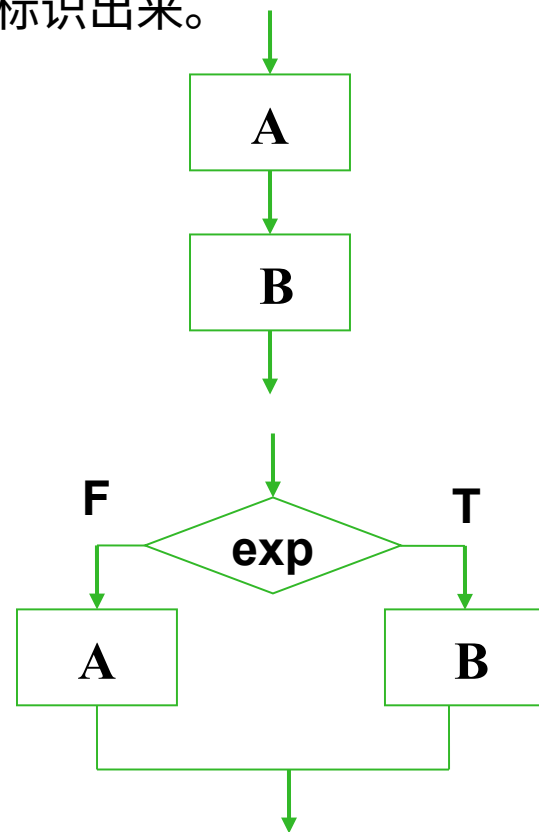
程序流程图又称程序框图，是用统一规定的标准符号描述程序运行具体步骤的图形表示。程序框图的设计是在处理流程图的基础上，通过对输入输出数据和处理过程的详细分析，将计算机的主要运行步骤和内容标识出来。

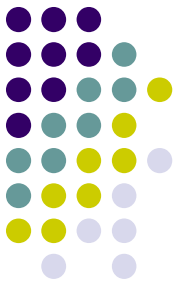
1、顺序

型
几个连续的加工依次序排列

2、选择

型
由某个判断式的取值决定选择两个加工中的一个。



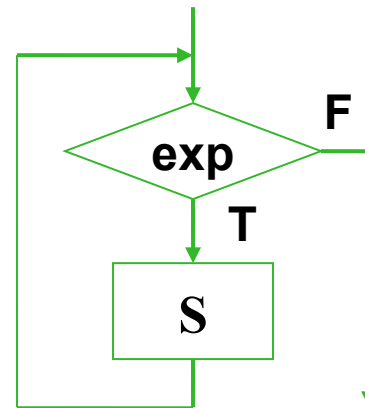


1. 结构化方法

1.3.2 程序流程图

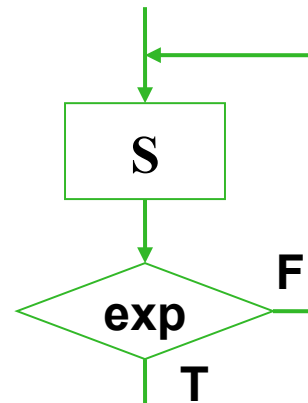
3、当型循环

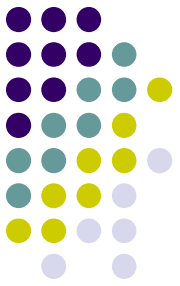
当循环控制条件成立时，
重复执行特定的加工。



4、直到型循环

重复执行特定的加工，直
到循环控制条件成立时。

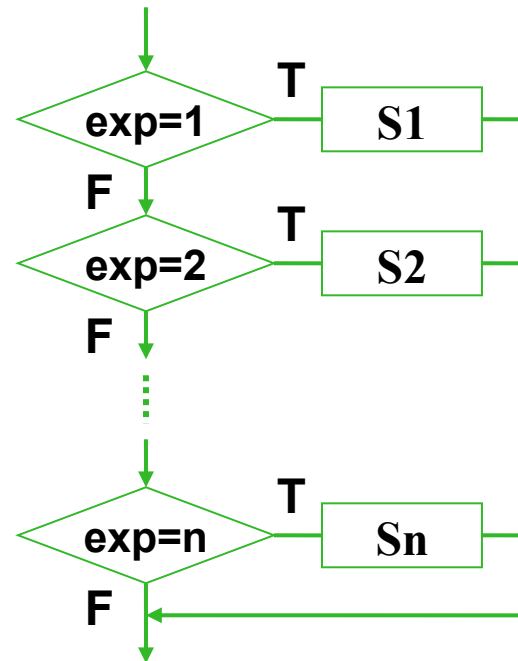




1. 结构化方法

1.3.2 程序流程图

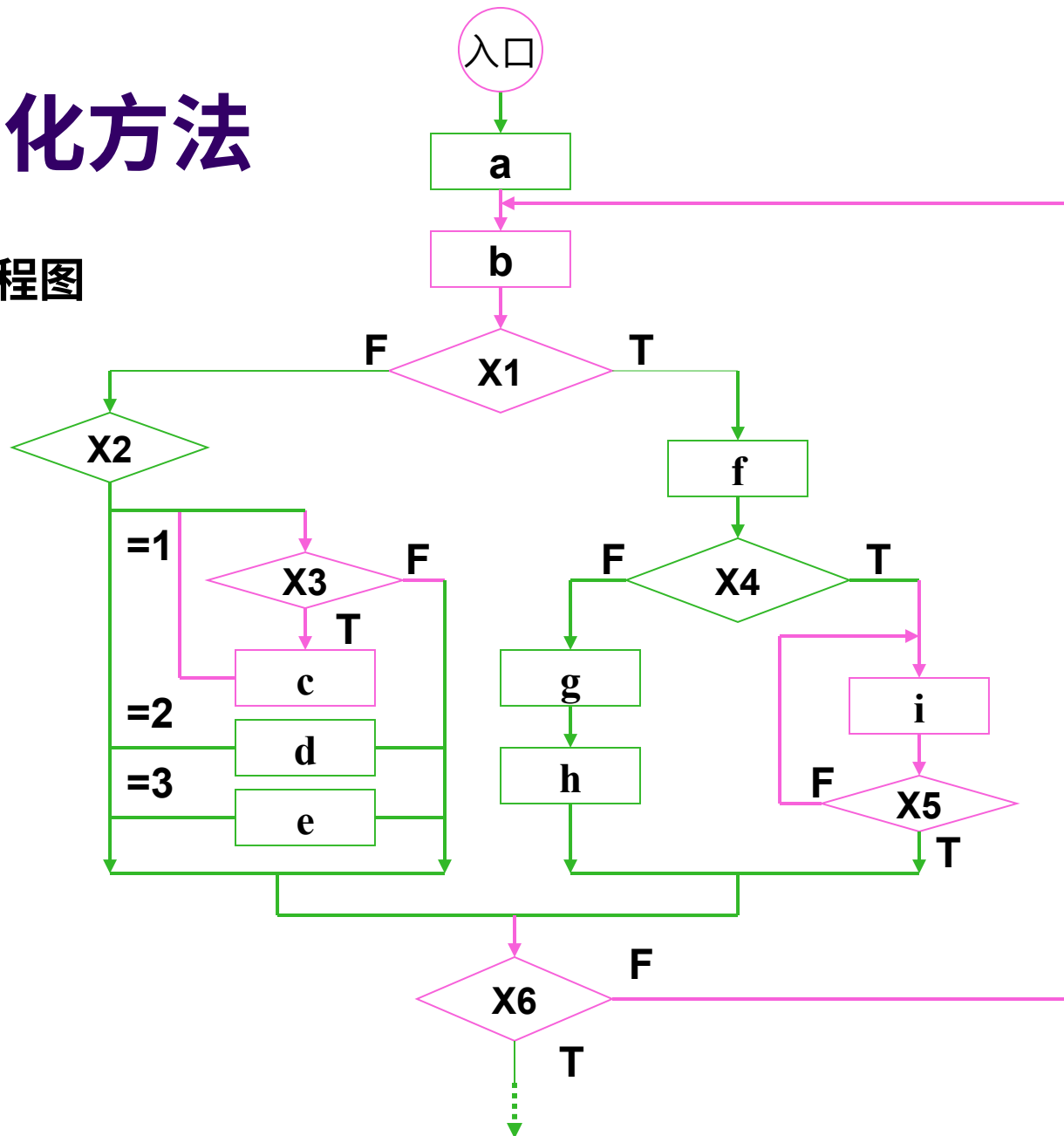
5、多情况选择
型
列出多种加工
情况，根据控制变
量的取值，选择执
行其一。





1. 结构化方法

1.3.2 程序流程图

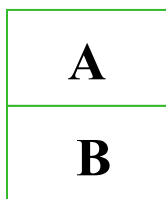




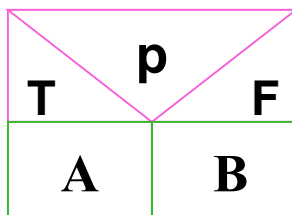
1. 结构化方法

1.3.2 N-S 图 —— Nassi and Shneiderman

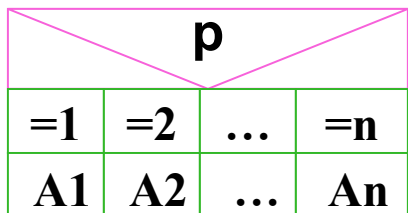
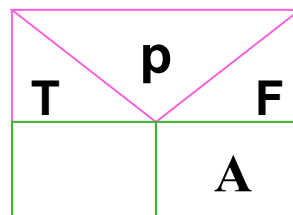
N-S 图，也被称为盒图或 NS 图，是结构化编程中的一种可视化建模



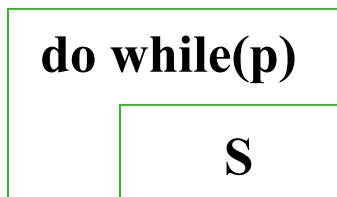
顺序型



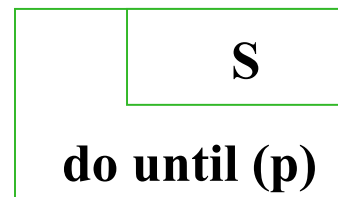
选择型



多分支选择型



当型循环型

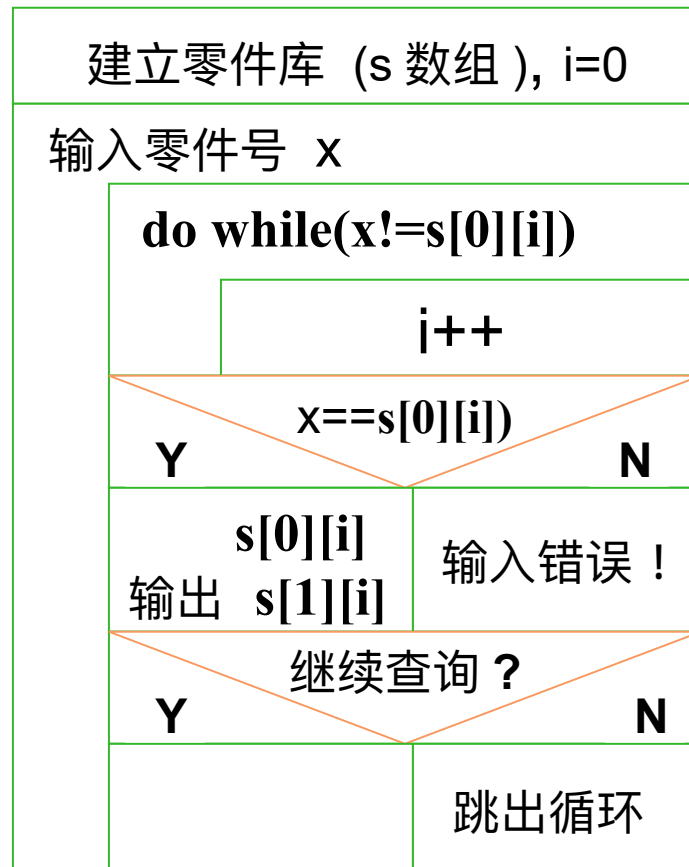


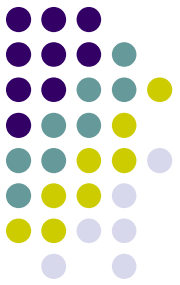
直到型循环型



1. 结构化方法

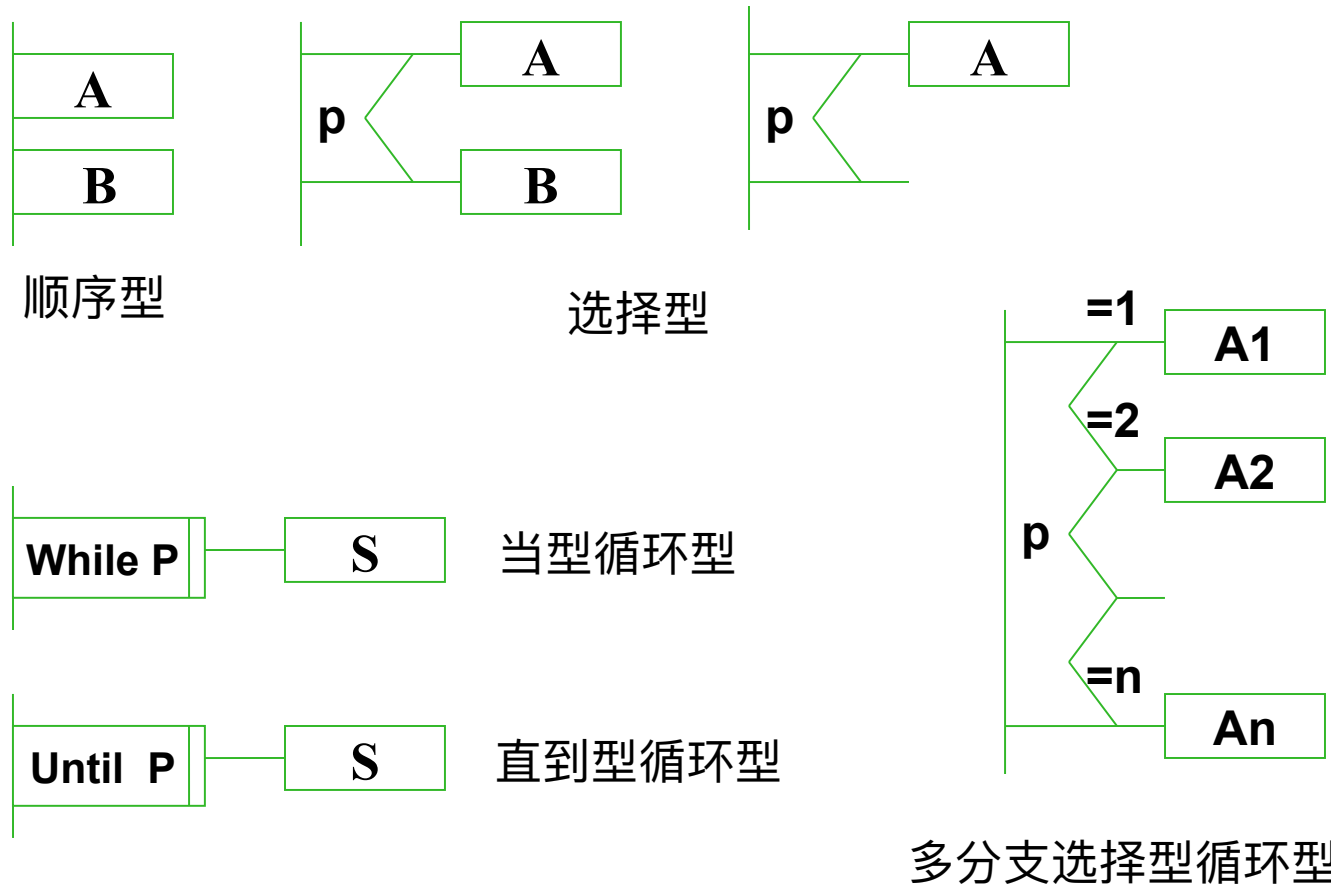
1.3.2 N-S 图 —— Nassi and Shneiderman





1. 结构化方法

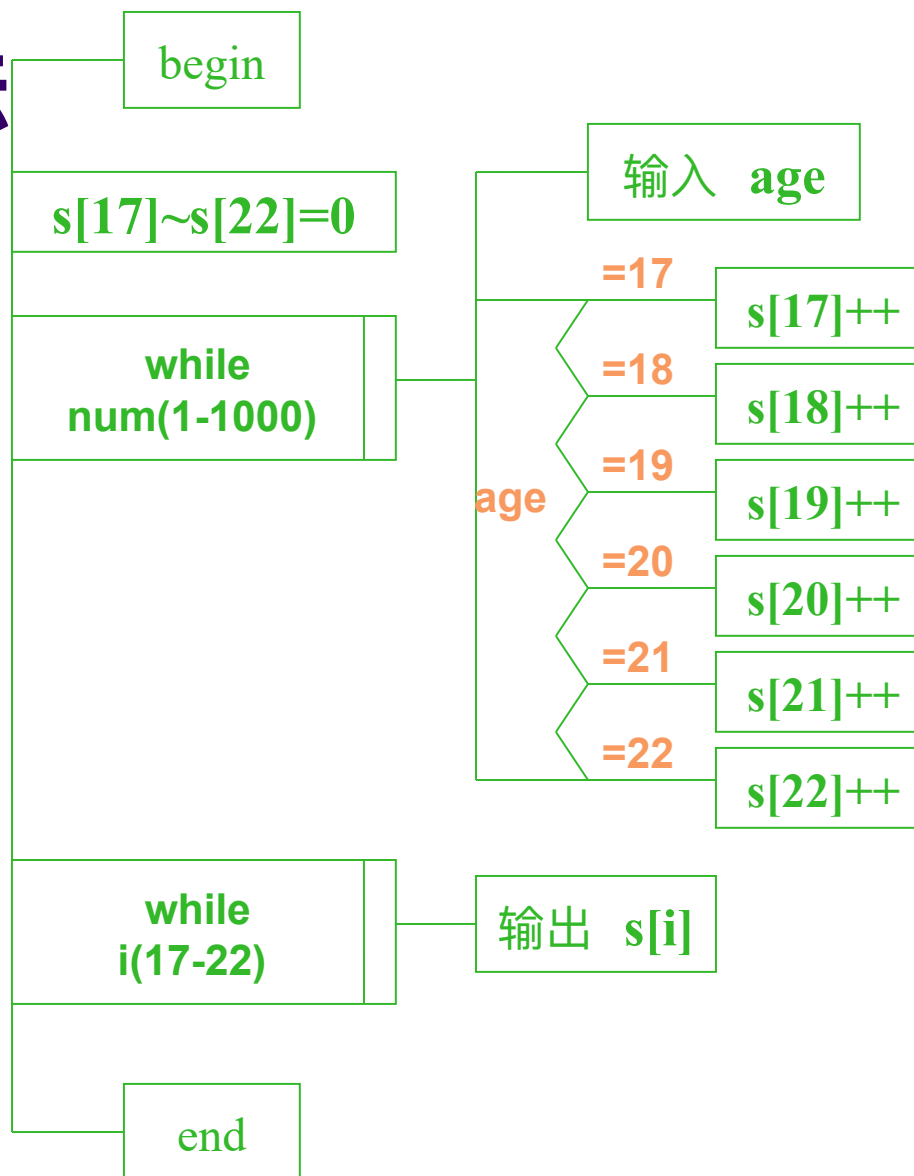
1.3.2 PAD 图 —— Problem Analysis Diagram



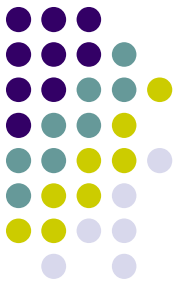


1. 结构化方法

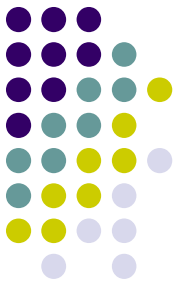
1.3.2 PAD 图



Two Approaches to Software Construction and Modeling



- The Structured Approach
 - Earlier approach. Assumes a system is a collection of processes that interact with data
 - Structured analysis, structured design, and structured programming
- The Object-Oriented Approach (面向对象方法)
 - More recent approach. Assumes a system is a collection of objects that interact to complete tasks
 - OO analysis, OO design, and OO programming

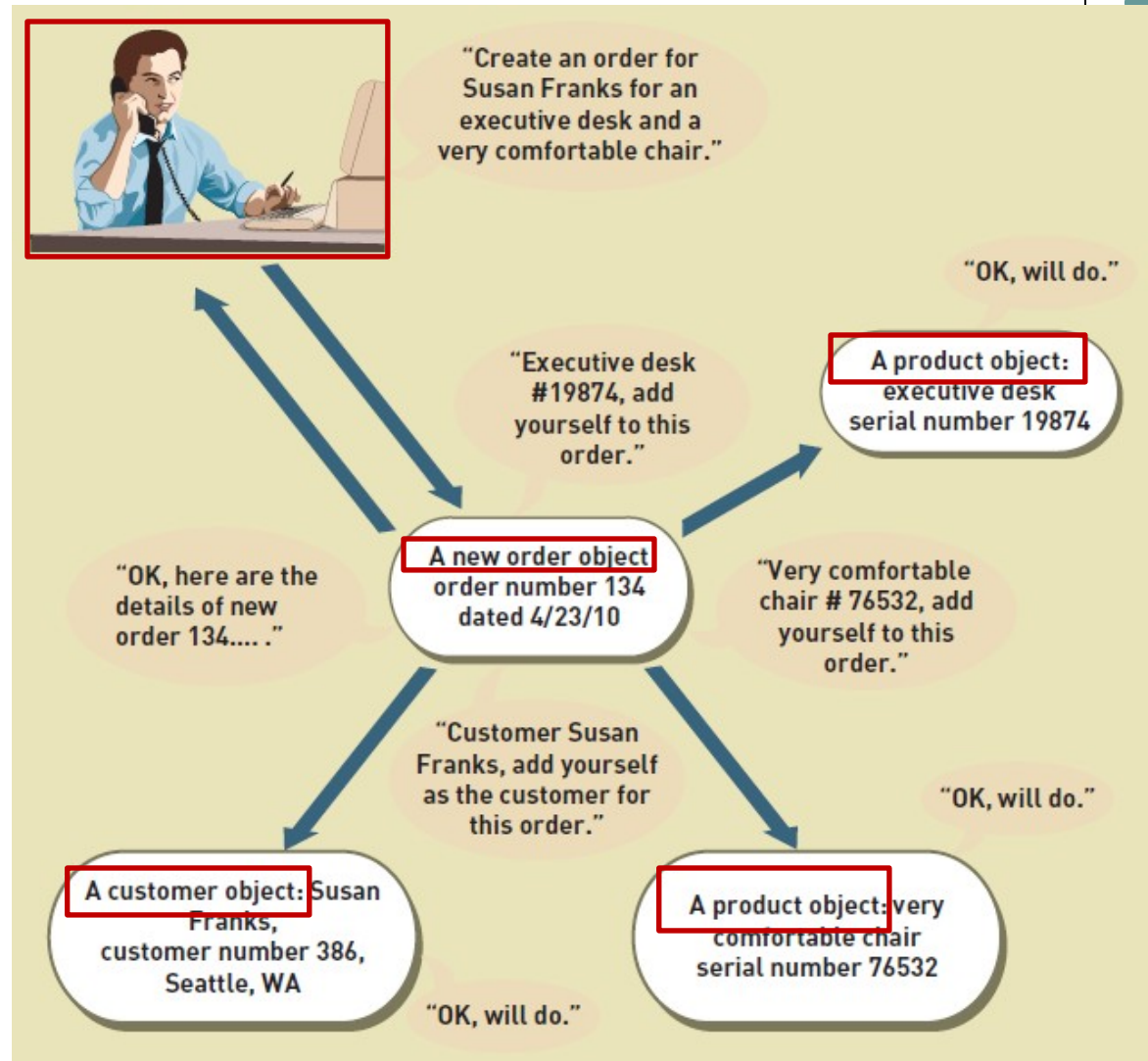


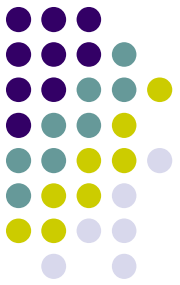
The Object-Oriented Approach

- Object-oriented analysis (OOA) 分析
 - The process of identifying and defining the use cases and sets of objects (classes) in the new system
- Object-oriented design (OOD) 设计
 - Defining all of the types of objects necessary to communicate with people and devices and showing how they interact to complete tasks
- Object-oriented programming (OOP) 编程
 - Writing statements that define the actual classes and what each object of the class does

The Object-Oriented Approach

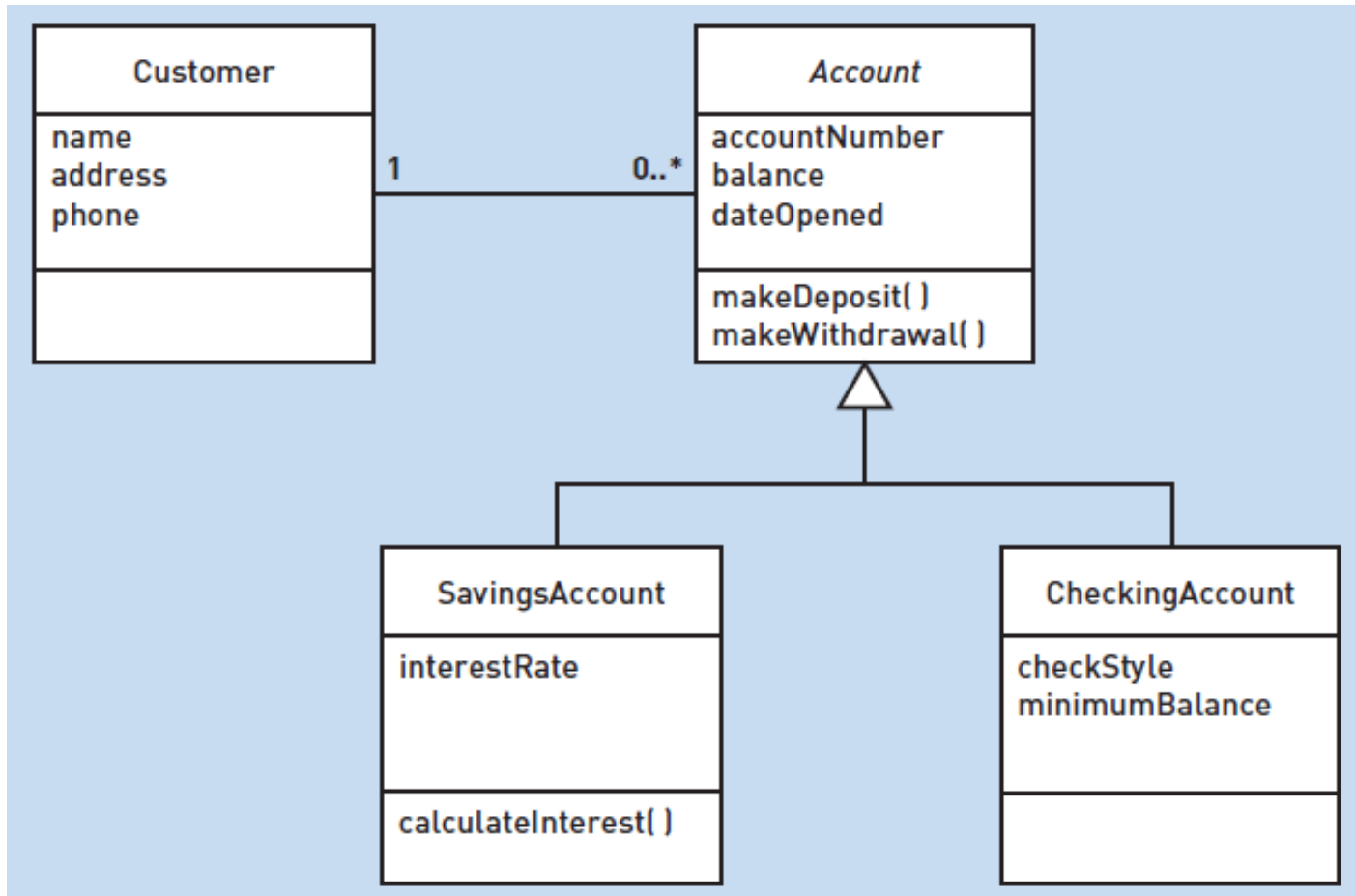
- Example showing the OO concept
- Objects collaborate to get a task done

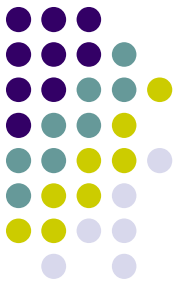




The Object-Oriented Approach

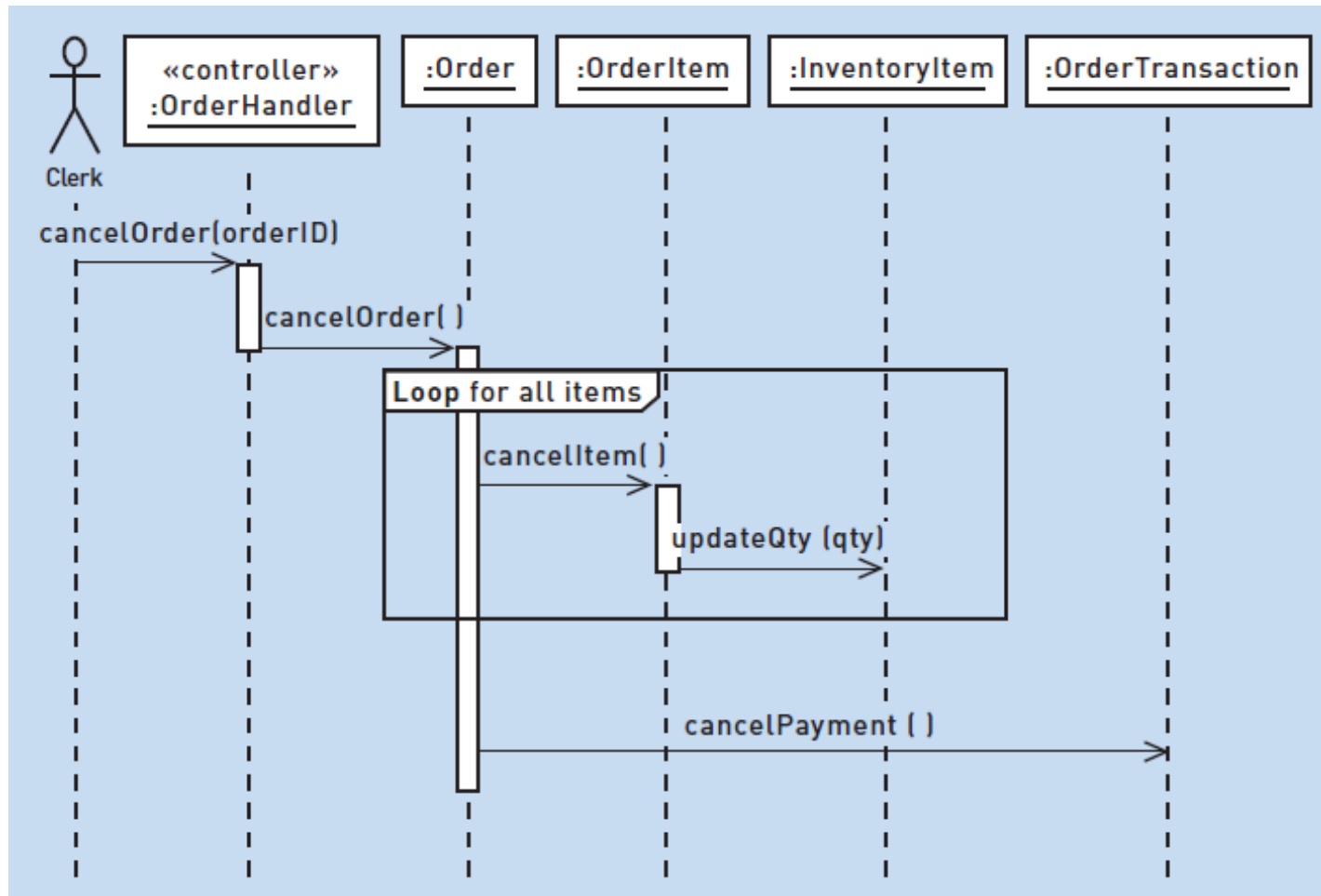
- UML Design Class Diagram (类图)



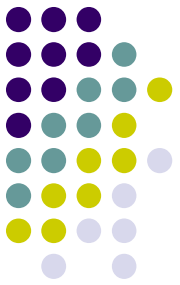


The Object-Oriented Approach

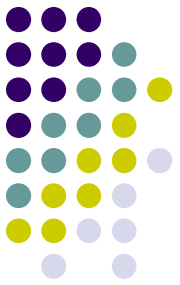
- UML Sequence Diagram (顺序图)



Summary

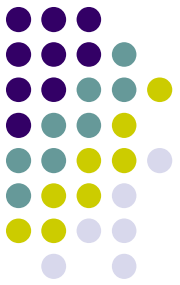


- This chapter covers approaches to system development in more detail (更详细地介绍系统开发方法)
- There are two approaches to the SDLC: Predictive (预测性) and Adaptive (适应性)
- A predictive SDLC, also known as the waterfall model, is used when it is possible to plan the project completely in advance
- An Adaptive SDLC, which uses iteration, is used when the requirements are less certain and the project will need to react to changes



Summary (continued)

- All new information systems require support once completed (所有新的信息系统一旦完成都需要支持)
- System development project use a methodology (or development process) and many are available. A methodology includes an SDLC and tools, techniques, and models (方法论包括工具、技术和模型)



Summary (continued)

- There are two approaches to construction and modeling software: the traditional structured approach and the newer object-oriented approach (结构化方法和面向对象的方法)
- Agile development is the current trend in system development (敏捷开发是现在的趋势)